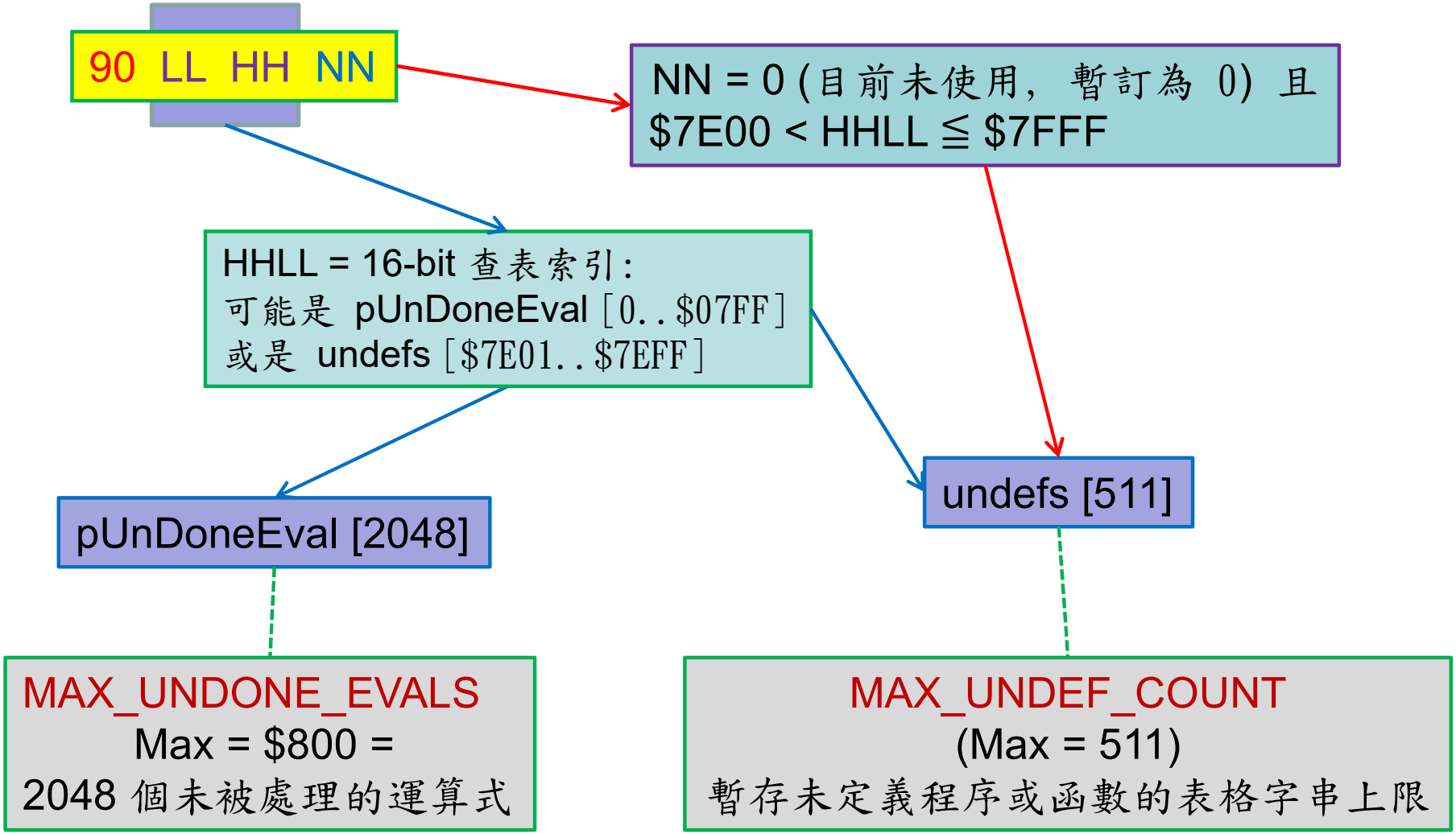
Helpful informations about “Revive Project”

Jurng Chen Su

蘇勇誠

FUNC_TOKEN = $90



VAR_TOKEN = $91

91 LL HH NN

HHLL = 16-bit 查表索引 ndx:
目前允許範圍 [0..$3FFF]
NN 目前保留未用，預設為 0

所有的變數總表 all_var [16384]

MAX_VAR_COUNT
Max = 16384 ($4000)
目前最多 16384 個變數

相關:
區域變數表 loc_list [locv_ndx] =
VAR_TOKEN | (ndx << 8)

all_var [ndx]

註:
loc_list [ ] 的上限 1024

SYMB_TOKEN = $92

92 CH LV XX

CH: 符號字元碼（查表 symbols[] , symbol2[])
LV: 數學運算等級,(15 最高,例如: 左右括號, 查表 symlevs[])
XX: 保留, 目前不使用, 填零

```
#define  TOKEN_LEFT_Bracket     (0x0F0300 | SYMB_TOKEN)
#define  TOKEN_EQUAL            (0x021400 | SYMB_TOKEN)
#define  TOKEN_RIGHT_Bracket    (0x0F0400 | SYMB_TOKEN)
#define  TOKEN_COLON            (0x031000 | SYMB_TOKEN)
#define  TOKEN_SEMICOLON        (0x036300 | SYMB_TOKEN)
```

```
char symbols[22] = { '.', '[', ']', '(', ')', '+', '-', '!',
                '*', '/', '%', '&', '^', '|', '~', '?',
                ':', ',', '>', '<', '=', ',' };
char *symbol2[26] = {"++", "--", "+", "-", "+=", "-=", ">=", ">>",
                ">>=", ">>>", ">>>=", "<=", "<<", "<<=", "<<<", "<<<=",
               "==", "!=", "*=", "/=", "%=", "&=", "^=", "|=",
               "&&", "||"   };
```

TYPE_TOKEN = $93

93 00 00 XX

XX: 保留，目前不使用，填零

TEMP_STR_TOKEN = $94

94 LL HH XX

HHLL: 暫時字串的索引（max: 127）
XX: 保留，目前不使用，填零

MIA files
ex. Answer.MIA
Tells to use blob file:
Test1234.blb

Block of binary (blob) files
ex. Test1234.blb
Contains TempStrList[..]

VarsMngr.pas

Runner.c
Stores the temporary strings
to array temp_str[128]

CONST_TOKEN = $95

95 LL HH XX

HHLL: 常數索引（index from 1）to the array of TMyConst
XX: 保留，目前不使用，填零

```
struct TMyConst {                // Constant 常數
        char    name [128];
        char    initVal [256];
        double  dVal;
        int     iVal;
        unsigned int     initNdx, line_no, prog_id, var_index;
        unsigned char    flag, vType;
};
```

RESV_TOKEN = $96

96 LL HH XX

HHLL: 保留字索引
XX: 保留，目前不使用，填零

查找 TOKEN_ID_LIST[HHLL]
對映文字 sys_cmd[HHLL]

```
enum TOKEN_ID_LIST { TOKEN_ID_BEGIN, TOKEN_ID_END,
        TOKEN_ID_REPEAT, TOKEN_ID_UNTIL, TOKEN_ID_IF,
        TOKEN_ID_THEN, TOKEN_ID_ELSE, TOKEN_ID_DO,
        TOKEN_ID_WHILE, TOKEN_ID_FOR, TOKEN_ID_DOWNTO,
        TOKEN_ID_TO, TOKEN_ID_STEP, TOKEN_ID_CASE,
        TOKEN_ID_OF, TOKEN_ID_CONTINUE,
        TOKEN_ID_BREAK, TOKEN_ID_EXIT  };
```

```
char *sys_cmd[MAX_SYS_CMD] = { "begin", "end", "repeat", "until",
        "if", "then", "else", "do", "while", "for", "downto", "to",
        "step", "case", "of", "continue", "break", "exit" };
```

BLOCK_TOKEN = $97

97 LL HH XX

HHLL: 切出程式小塊索引
XX: 保留，目前不使用，填零

exec_this_small_block()

RES_FUNC _TOKEN = $98

98 LL HH XX

HHLL: 切出內建函數索引= func_id
XX: 保留，目前不使用，填零

do_run_resi_func(func_id)

pLocals = prepare_local_vars(pPF);
// 預備此次呼叫的區域變數
parse_arguments(pPF, func_id, pLocals);
// 剖析後面的參數到適當的區域變數中
run_resident_func(func_id, pLocals);
// 從外部來執行一個內建函式

```
struct TFuncProc resident_funs[3] = {
        { "printf",    0, 0, 4, NULL}, // !! 此處參數應是可變的 !!
        { "return",    0, 0, 1, NULL},
        { "無此函數", 0, 0, 0, NULL} };
```

```
char *Resident_FuncName[RES_FUNC_COUNT] = { “printf”, "return" };
        // for test !
```

STRU _TOKEN = $99

99 LL HH XX

HHLL: 切出結構索引
XX: 保留，目前不使用，填零

Strus[ ]

LineNo_TOKEN = $9E

9E LL HH PP

HHLL:本行程式所含有的 Token 數量
PP: 程式編號

LineEnd_TOKEN = $9F

9F LL HH PP

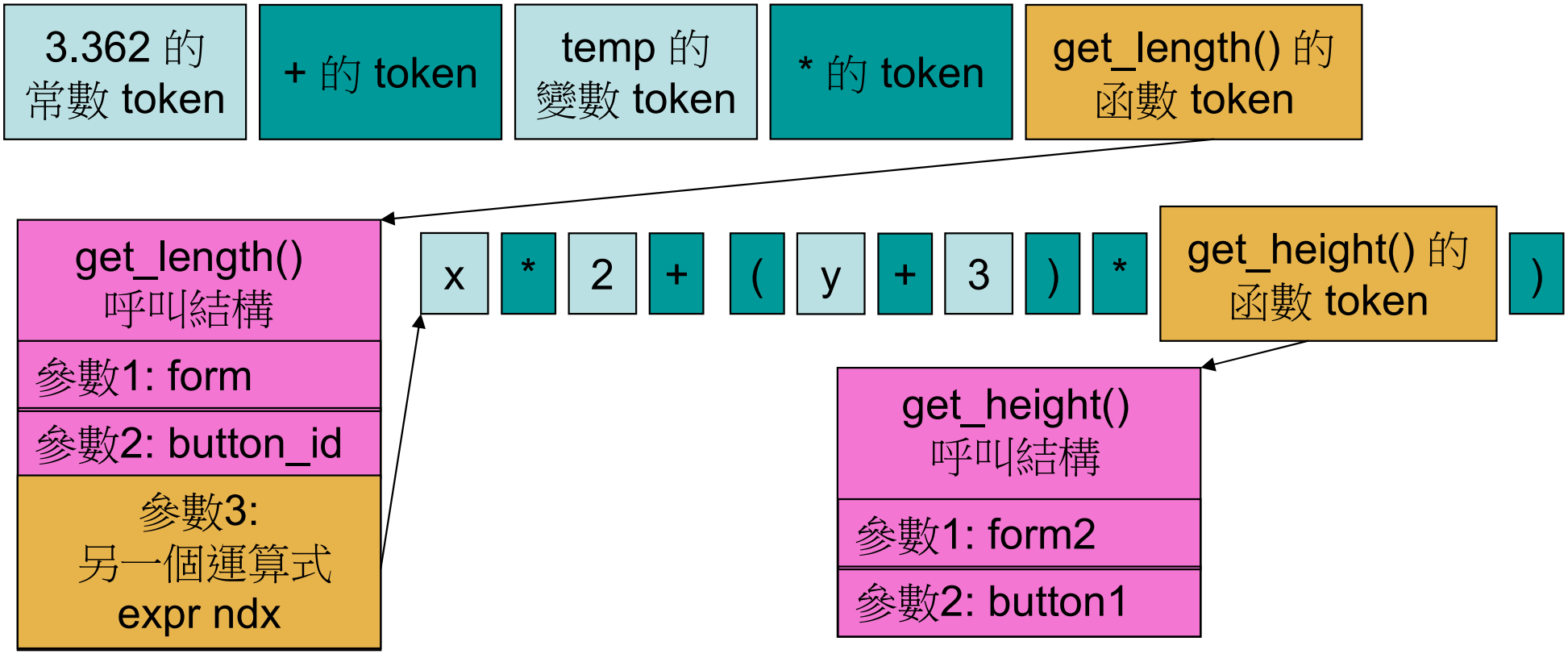
HHLL: 行號
PP: 程式編號 (.scp files 執行腳本檔索引)

對於 expr 表示式裏面函式的處理

假設下面是要處理的一行數學運算式：

3.362 + temp * get_length(form, button_id, x*2 + (y+3)*get_height(form2, button1))

則經過第一道處理, 會得到：

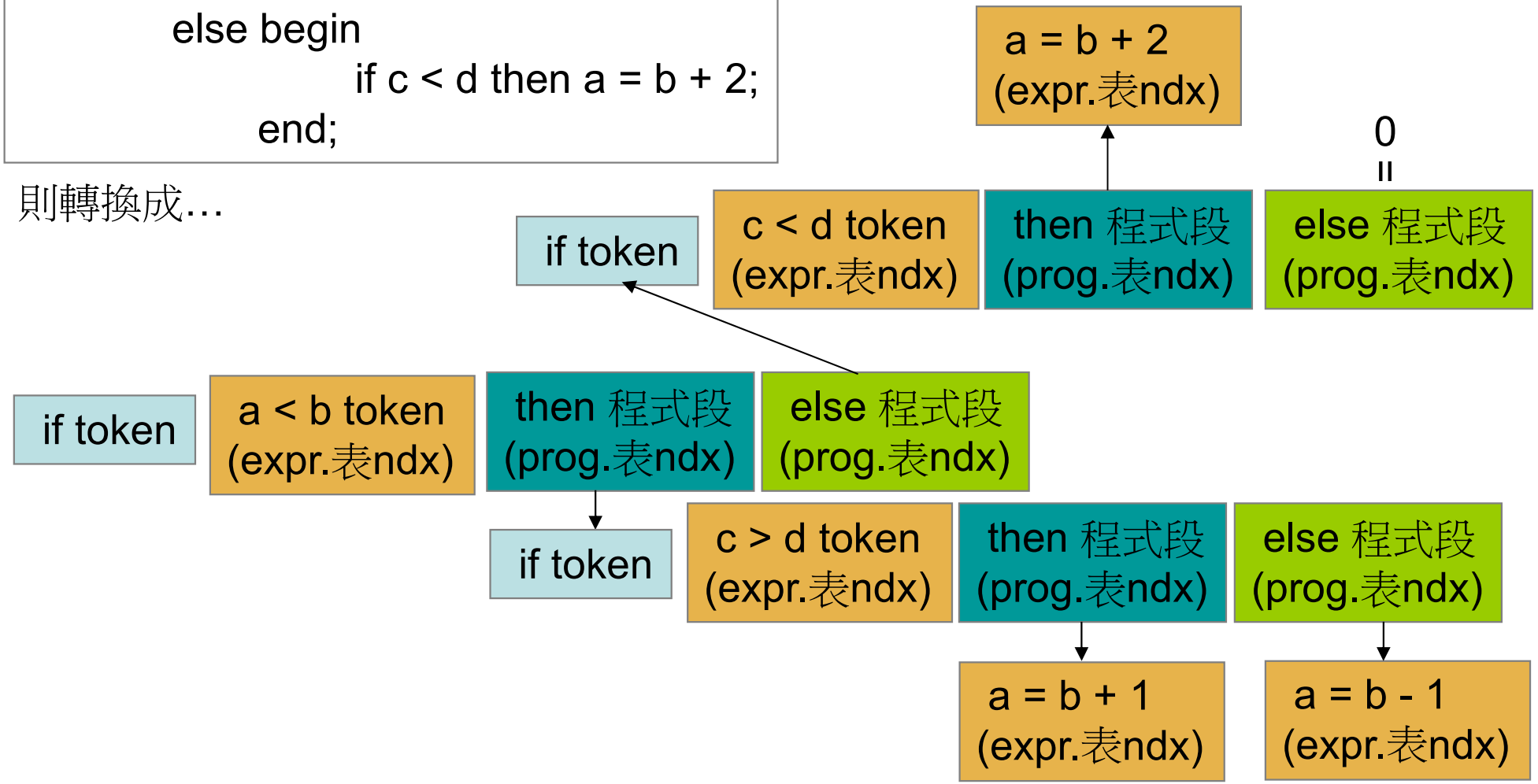


對於 if (expr) then [..] else [..] 的處理

假設下面是要處理的程式..

```
 if a < b then begin
                    if c > d then a = b + 1
                                 else a = b – 1;
            end
        else begin
                    if c < d then a = b + 2;
            end;
```

則轉換成…



If .. then .. Else .. 例子

- 9E 06 00 00 - 96 04 04 01 - 91 03 00 00 - 92 14 02 00 - 95 06 00 00

  if y = 12 (行號 20 已經不見)

  → 04 代表 [96 04 04 01] 這個 if token 距離下一個 then token [96 05 04 01] 為 4 個 tokens.
- 96 05 04 01-97 04 00 00 - 9F 16 00 00

  then [程式小段 04]

  → 04 代表 [96 05 04 01] 這個 then token 距離前一個 if token [96 04 00 01] 為 4 個 tokens. (9F16 00 00 是行號 22)
- 9E 02 00 00 - 96 06 00 01 - 97 02 00 00 - 9F 1C 00 00

  else [程式小段 02] (9F1C 00 00 是行號 28)

數學運算之數值類型轉換

原則上, 以轉成左側同類為優先, 有正負號傾向保留符號

L / R	0 array	1 char	2 byte	3 word	4 dword	5 sint8	6 sint16	7 sint32	8 dblchar	9 unicode	10 String	11 single	12 Double	13 pchar	14 ptr	15 sint64r	16 qword
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	5	5	7	7	7	7	7	48	49	50	47	47	4	4	47	44
2	0	4	4	4	4	7	7	7	48	49	50	47	47	4	4	47	44
3	0	4	4	4	4	7	7	7	48	49	50	47	47	4	4	47	44
4	0	4	4	4	4	7	7	7	48	49	50	47	47	4	4	47	44
5	0	7	7	7	7	7	7	7	48	49	50	47	47	7	7	47	47
6	0	7	7	7	7	7	7	7	48	49	50	47	47	7	7	47	47
7	0	7	7	7	7	7	7	7	48	49	50	47	47	7	7	47	47
8	0	8	8	48	48	48	48	48	8	8	8	48	48	48	48	48	48

型態[0..16] = ('array', 'char', 'byte', 'word', 'dword', 'sint8', 'sint16', 'int', 'dblchar', 'unicode', 'string', 'single', 'double', 'pchar', 'ptr', 'integer', 'sint64', 'qword');

數學運算之數值類型轉換

原則上, 以轉成左側同類為優先, 有正負號傾向保留符號

L / R	0 array	1 char	2 byte	3 word	4 dword	5 sint8	6 sint16	7 sint32	8 dblchar	9 unicode	10 String	11 single	12 Double	13 pchar	14 ptr	15 sint64r	16 qword
9	0	9	9	49	49	9	9	49	9	9	9	89	89	49	49	89	89
10	0	10	10	50	50	10	10	50	10	10	10	90	90	10	10	90	90
11	0	11	11	11	11	11	11	11	91	91	91	11	12	51	51	51	51
12	0	12	12	12	12	12	12	12	92	92	92	12	12	52	52	52	52
13	0	D	D	D	D	D	D	D	0	0	0	47	47	D	D	53	53
14	0	7	4	4	4	7	7	7	0	0	0	47	47	14	14	54	54
15	0	15	15	15	15	15	15	15	95	95	95	55	55	55	55	15	15
16	0	16	16	16	16	16	16	16	96	96	96	56	56	56	56	16	16

= 0 的值代表兩邊都不用動他！(免轉換)

> 80 的值要扣去 80, 並產生 <無法預期轉換結果是否仍然有效的警告>

> 40 的值要扣去 40, 並產生 <轉換結果可能有疑慮的警告> (數值或符號損失)